TP intégrateur DevSecOps et AIOps

TECHCORP, Orders API : sécuriser la livraison et maîtriser l'incident INC-2026-1042

Équipe	Paul Chavanon, Nicodem BAJOR
Formation	CFA INSTA, RNCP niveau 7, DevOps et IA
Date	22 octobre 2026
Dépôt Git	github.com/Paul-CHAVANON/techcorp-devsecops-aiops
Environnement	MacBook Apple Silicon, Docker Desktop 29.9.2, Python 3.12, Ollama avec qwen3:14b

> Outils : l'exécution et la rédaction ont été assistées par un assistant IA. Les choix techniques, la vérification des résultats et l'approbation de la remédiation ont été faits par l'équipe.

Sommaire et conventions

Ce rapport suit les missions du sujet : A (pipeline DevSecOps), B (observabilité et détection), C (investigation AIOps) et D (revue SRE). Chaque capture porte un bandeau avec l'URL affichée et l'heure de la prise. Les heures sont en heure de Paris (UTC+2), sauf celles des logs et des rapports, qui sont en UTC.

Le détail de l'exécution, avec les problèmes rencontrés et leurs corrections, est dans `docs/journal-execution.md` du dépôt.

Mission A : pipeline DevSecOps

A1. Registre des risques

Nous avons relevé cinq risques, cotés en gravité et probabilité de 1 à 4, chacun avec une mesure corrective et le contrôle qui la vérifie.

#	Risque	Criticité	Mesure
R1	Secret exposé dans le dépôt ou l'historique Git	12	Gitleaks sur tout l'historique, bloquant à chaque PR
R2	Dépendance transitive vulnérable (starlette 0.46.2)	9	Montée de version, Trivy fs et image
R3	Image conteneur non maîtrisée	9	Healthcheck, cap_drop, setuptools 84.0.0, Trivy image
R4	Chaîne d'approvisionnement CI (actions par tag)	8	Actions épinglées par SHA, quality gate obligatoire
R5	Agent AIOps détourné ou trop privilégié	8	Logs traités comme données non fiables, masquage, aucun outil pour le LLM

Le risque R3 s'est confirmé pendant le TP : Trivy a trouvé deux vulnérabilités HIGH dans le setuptools livré avec l'image de base. Détail complet : `docs/registre-risques.md`.

A2. Pipeline de sécurité

Le workflow `.github/workflows/ci.yml` enchaîne cinq jobs : tests Pytest et promtool, Gitleaks sur tout l'historique, Trivy fs, construction de l'image puis Trivy image, et enfin le job « Quality gate ». Les actions sont épinglées par SHA, les scanners tournent dans des images à version fixe et le GITHUB_TOKEN est limité à `contents: read`.

Le premier run sur main a été rouge. Trivy image bloquait sur CVE-2026-23949 (jaraco.context 5.3.0) et CVE-2026-24049 (wheel 0.45.1), deux composants embarqués dans setuptools. Un correctif existait, donc nous avons corrigé au lieu de demander une exception : installation de setuptools 84.0.0 dans le Dockerfile (PR #1). Aucun contrôle n'a été affaibli, et le run suivant était vert.

A3. Quality gate

Ce qui bloque automatiquement la PR :

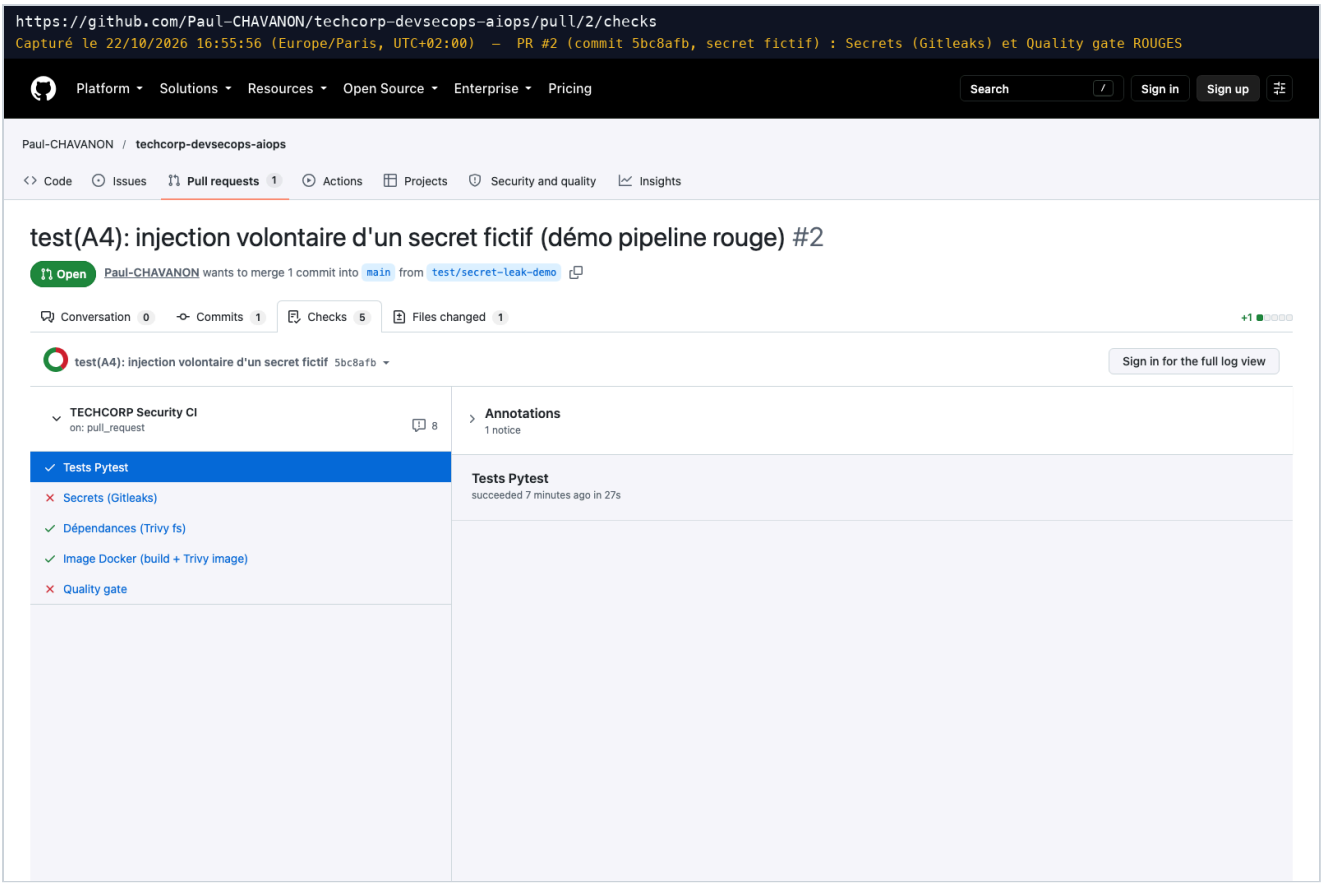
- un test en échec ;
- un secret détecté ;

- une vulnérabilité HIGH ou CRITICAL avec correctif disponible, dans les dépendances ou dans l'image.

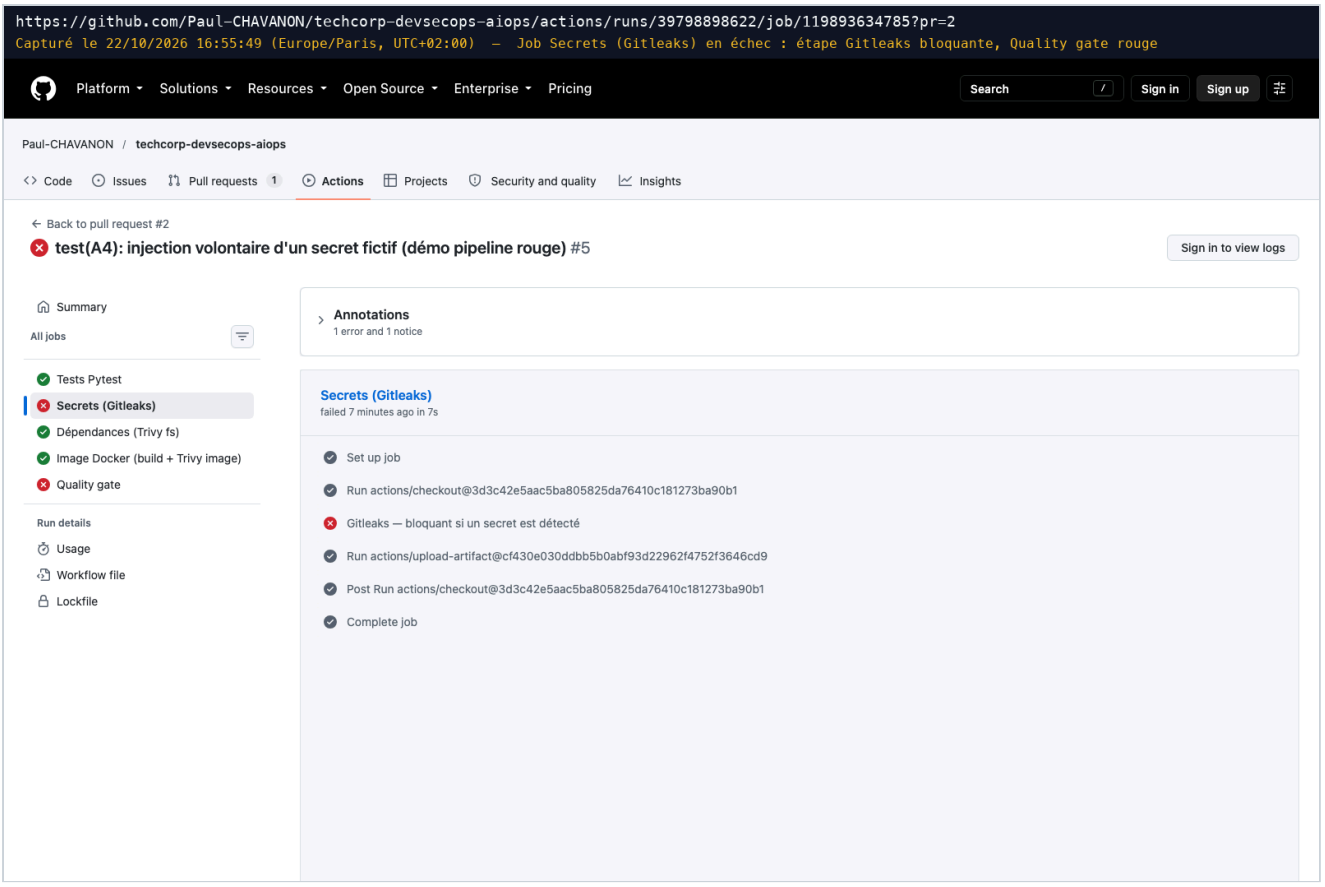
Nous distinguons trois cas. Une vulnérabilité exploitable se corrige. Pour un faux positif, on corrige la règle de détection, jamais l'outil. Une exception approuvée doit être justifiée, nommée et datée. La branche main est protégée : le check « Quality gate » et une PR sont obligatoires, administrateurs compris. Détail : `docs/quality-gate.md`.

A4. Pipeline rouge puis vert

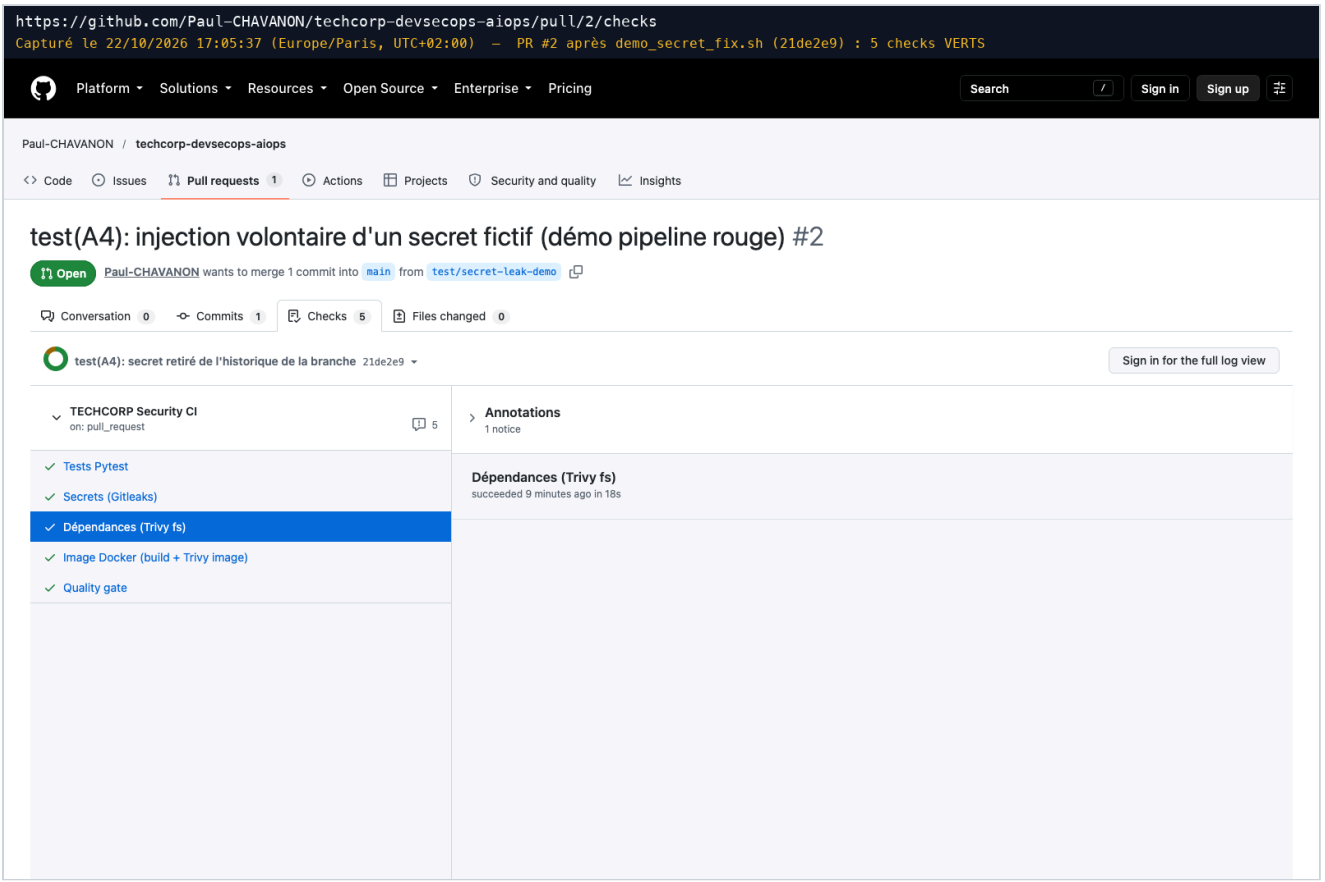
Le script `demo_secret_leak.sh` ajoute un marqueur de secret fictif sur une branche de test (PR #2). Gitleaks le détecte, et le quality gate bloque la fusion. Un simple `git rm` ne suffit pas, puisque Gitleaks lit tout l'historique. `demo_secret_fix.sh` réécrit donc la branche, et les cinq checks repassent au vert. La PR a ensuite été fermée sans fusion.



16:55:56. PR #2, commit 5bc8afb : « Secrets (Gitleaks) » et « Quality gate » en échec.



16:55:49. Détail du job : l'étape Gitleaks bloquante échoue.



17:05:37. Même PR après réécriture de l'historique (commit 21de2e9) : les 5 checks sont verts.

A5. Secrets et permissions

Les secrets GitHub Actions sont chiffrés, injectés seulement au moment du job et masqués dans les logs. Ce masquage n'est pas une protection complète : un simple encodage du secret le contourne. Le GITHUB_TOKEN doit n'avoir que les permissions nécessaires (ici lecture seule).

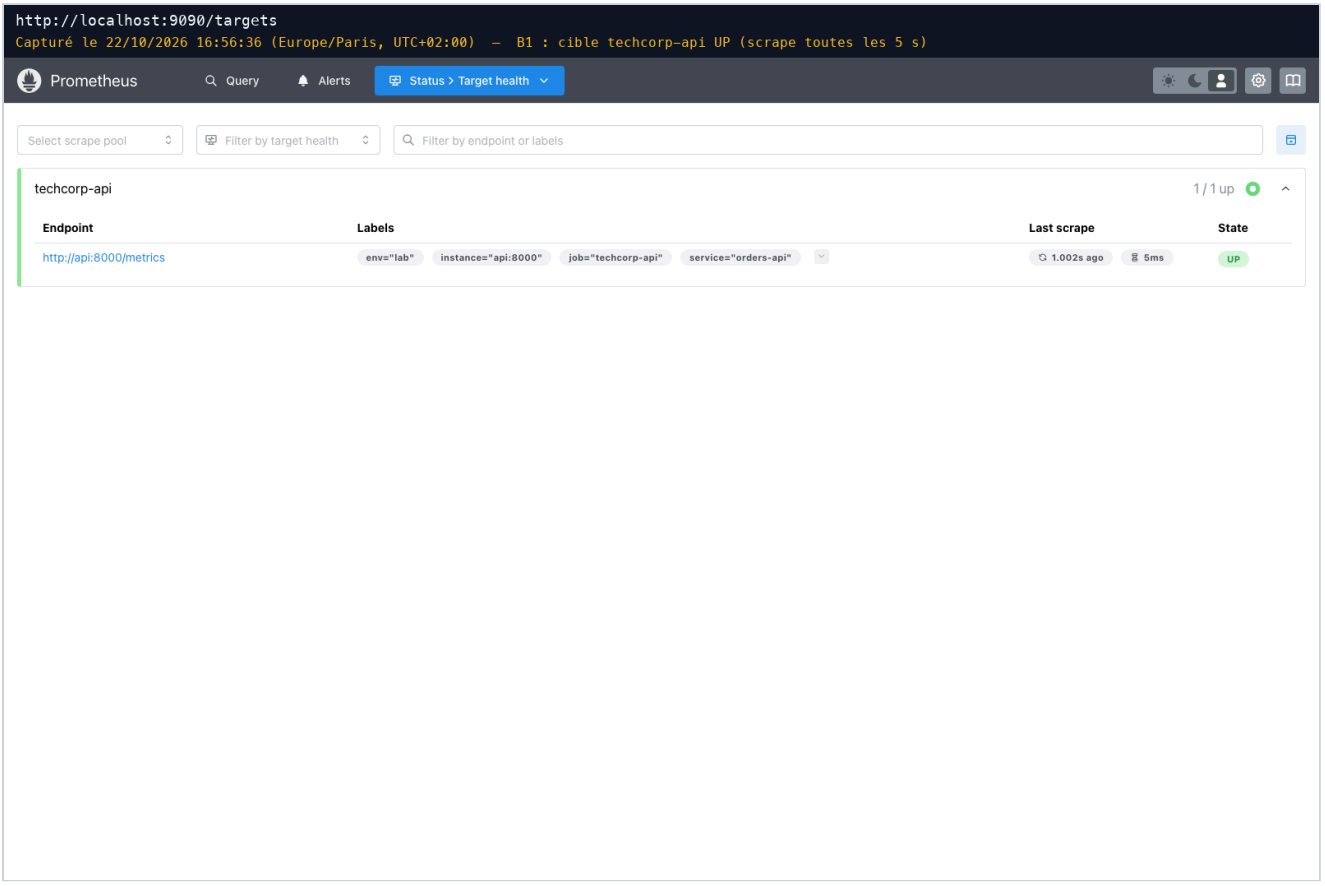
Un Secret Kubernetes n'est qu'encodé en base64. Il est stocké en clair dans etcd si le chiffrement au repos n'est pas activé, et toute personne qui a le droit de lire les secrets du namespace peut le lire. Il lui manque ce qu'apporte un vrai coffre : audit des accès, rotation, secrets à durée de vie courte. Détail : `docs/secrets-et-permissions.md`.

Mission B : observabilité et détection

B1. Cible Prometheus et séries

La cible techcorp-api est UP, avec une collecte toutes les 5 secondes. Les requêtes PromQL utilisées sont listées ci-dessous.

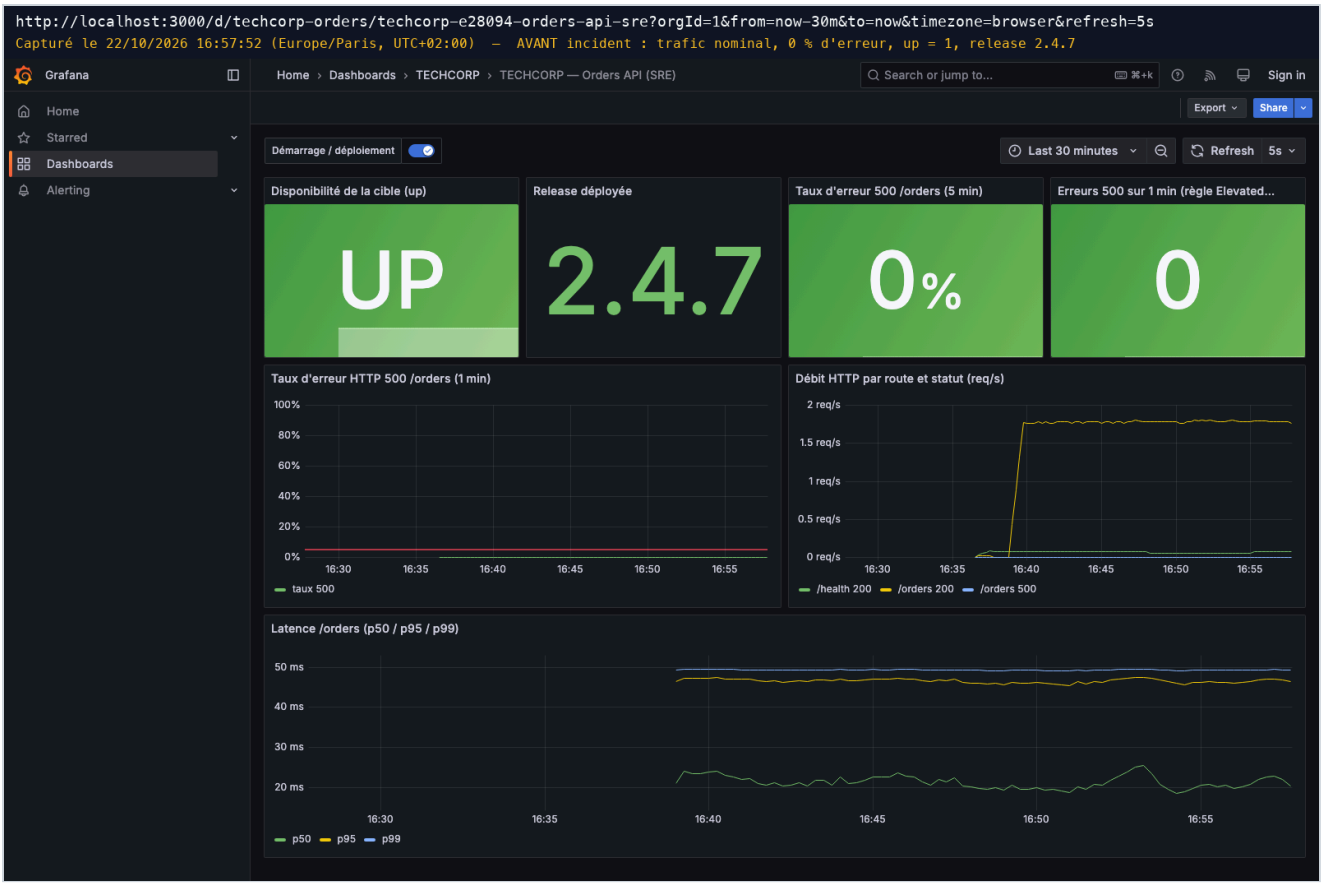
Objectif	PromQL
Disponibilité	`up{job="techcorp-api"}`
Requêtes 200 et 500 par seconde	`sum(rate(techcorp_http_requests_total{path="/orders",status="500"}[1m]))`
Taux d'erreur	`sum(rate(...{status="500"}[1m])) / clamp_min(sum(rate(...[1m])), 1e-9)`
Latence p95	`histogram_quantile(0.95, sum by (le) (rate(techcorp_request_duration_seconds_bucket[1m])))`



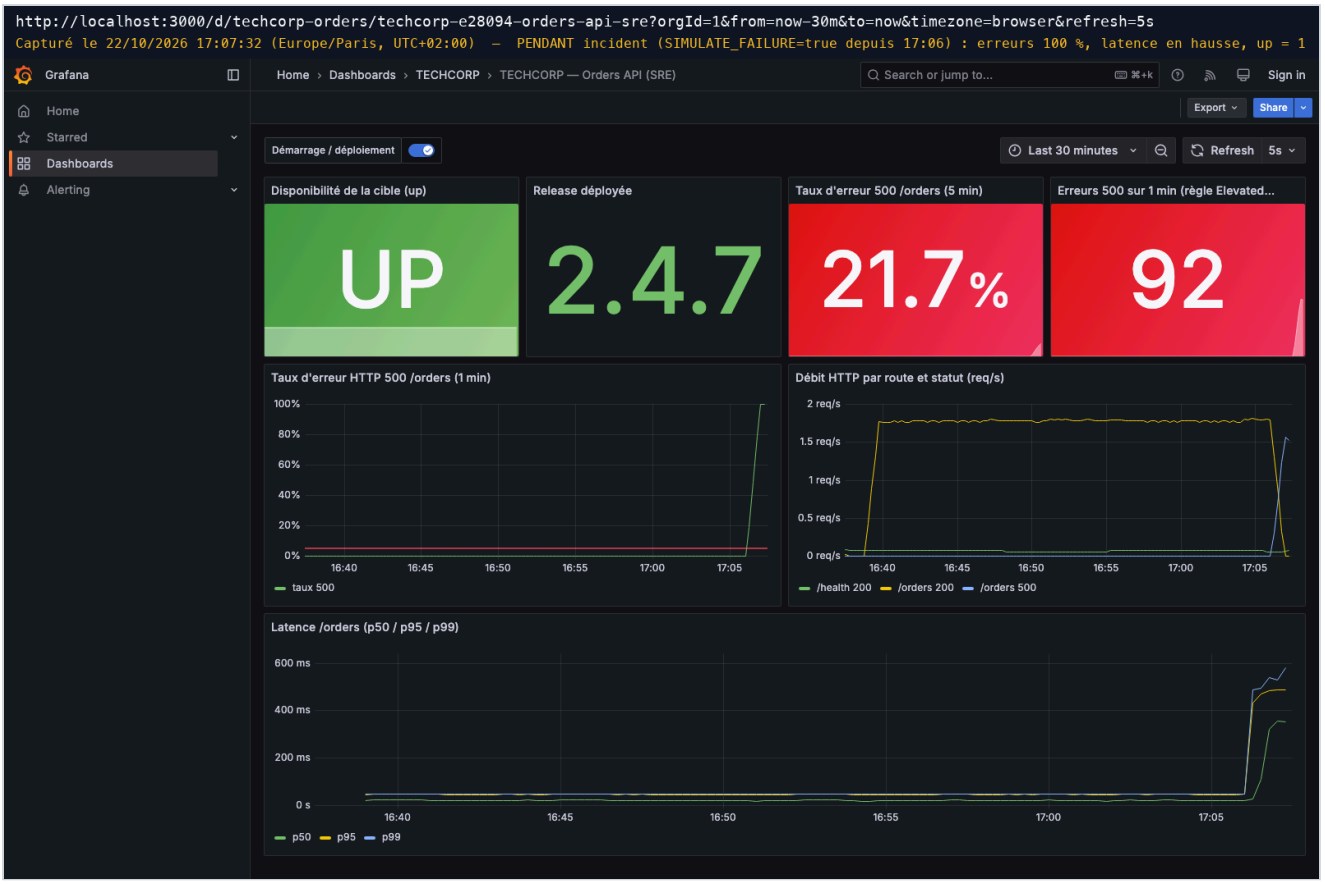
16:56:36. Prometheus, page Target health : techcorp-api est UP.

B2. Tableau de bord Grafana

Le dashboard est provisionné automatiquement. Il affiche la disponibilité, la release, le taux d'erreur sur 5 minutes et sur 1 minute (seuil à 5 %), le nombre d'erreurs 500 de la règle fournie, le débit par route et la latence p50, p95 et p99.



16:57:52. Avant l'incident : environ 1,8 requête par seconde, 0 % d'erreur, p95 vers 47 ms.



17:07:32. Pendant l'incident : 100 % d'erreur sur 1 minute, p95 vers 500 ms. La cible reste UP : le processus répond, mais le service est inutilisable.

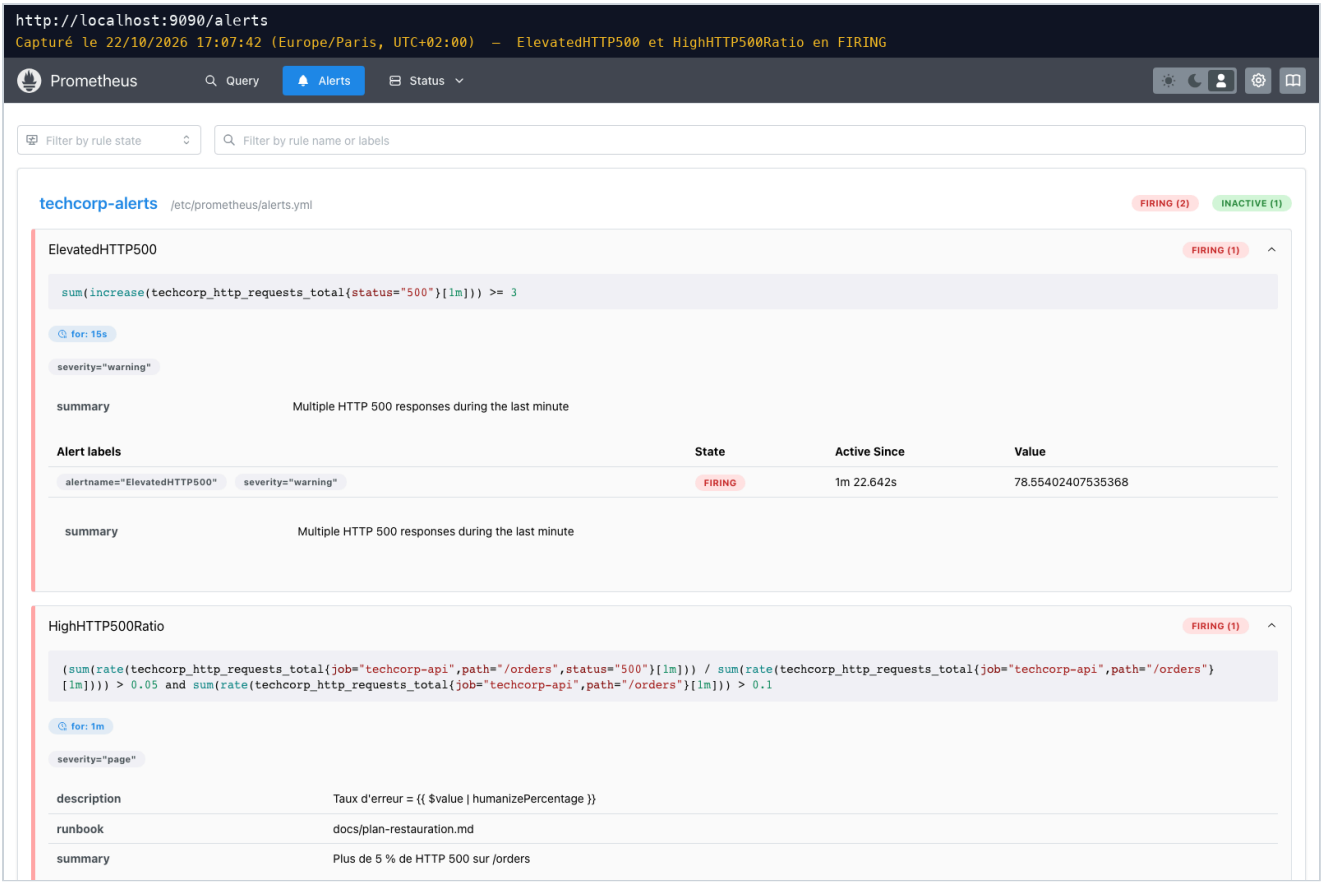
B3. Incident de laboratoire

Le script `simulate_incident.sh 30` recrée l'API avec `SIMULATE_FAILURE=true` et fait 30 appels à /orders, qui reçoivent tous une erreur 500. L'incident a été lancé à 17:06, puis rejoué à 17:37 pour produire un rapport à la chronologie juste (voir C1).

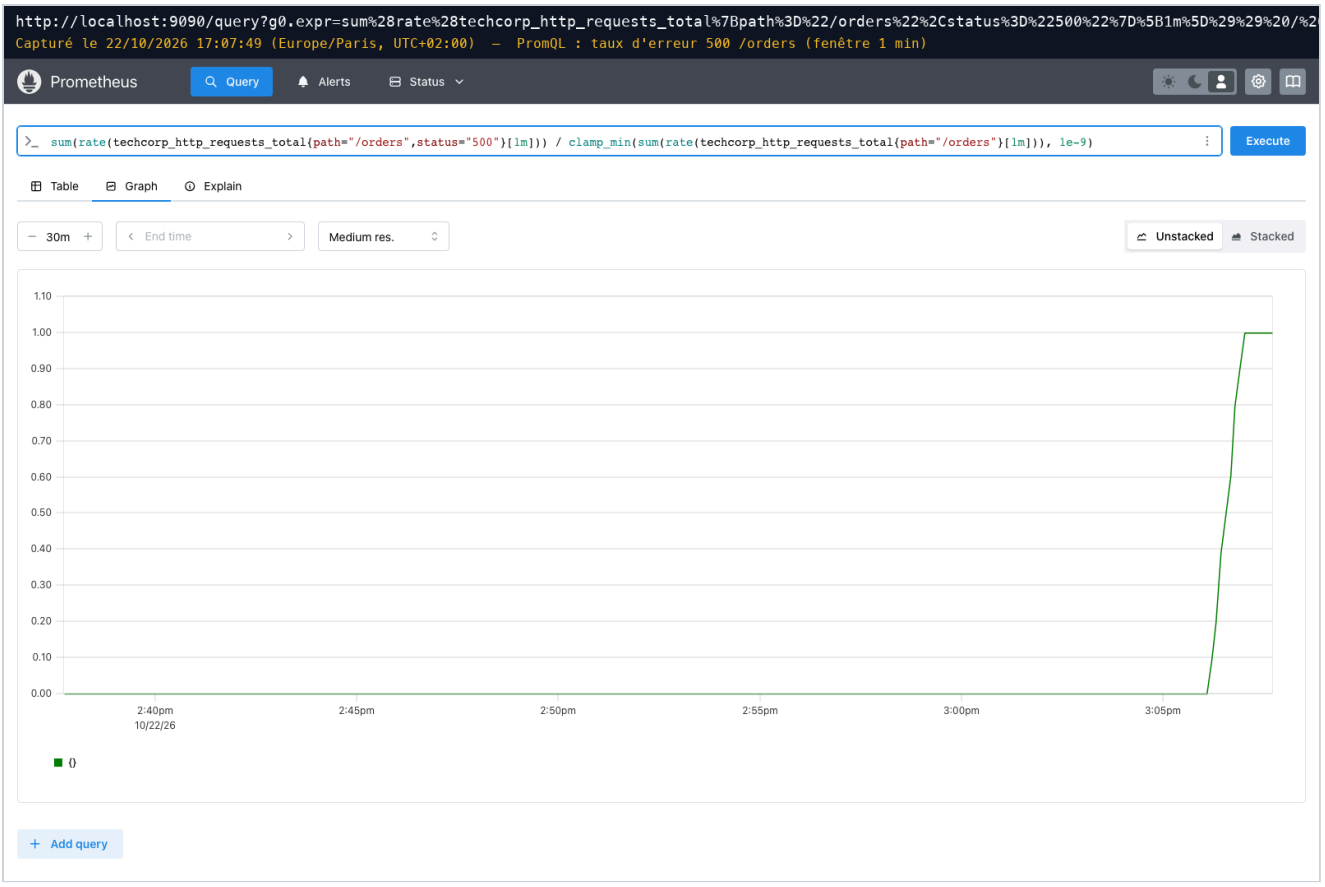
B4. Règle ElevatedHTTP500 et détection d'anomalies

La règle fournie compte un nombre absolu d'erreurs. Trois erreurs sur trois requêtes ou sur 30 000 déclenchent donc la même alerte. Nous avons ajouté la règle HighHTTP500Ratio, qui raisonne en proportion avec un trafic minimum. Les deux règles sont testées par promtool dans la CI.

Un seuil répond à la question « est-ce au-dessus de X ? ». Une détection statistique répond à « est-ce anormal pour ce service, par rapport à son historique ? ».



17:07:42. ElevatedHTTP500 et HighHTTP500Ratio en Firing (actives depuis 15:06:11 UTC).



17:07:49. Graphe PromQL du taux d'erreur sur /orders, qui passe de 0 à 1 (axe en UTC).

B5. Détecteur

Le détecteur (`aiops/detector.py`) calcule le taux d'erreur sur les 5 dernières minutes. Il le compare à une baseline, c'est-à-dire la moyenne et l'écart-type des minutes précédentes, et en tire un z-score et un ratio. Le verdict est ANOMALY si le taux dépasse 5 % et que le z-score ou le ratio dépasse 3. Chaque étape du calcul est écrite dans le champ `reasons`, et le code retour vaut 2 en cas d'anomalie. Le détecteur est couvert par 10 tests, sur 25 tests au total dans le projet.

Heure	Verdict	Lecture
17:08	ANOMALY	Taux de 33,5 %, baseline à 0 %, z = 67. Cas classique.
17:38	OK	Pendant le rejeu, la baseline contenait encore le premier incident. Le z-score tombait à -0,22, alors que les alertes Prometheus étaient en Firing.
17:42	ANOMALY	C'est le ratio (3,07) qui déclenche la détection, le z-score n'étant que de 1,33.

Mission C : investigation AIOps

C1. Corrélation avec le changement

Le collecteur rapproche la première erreur vue dans les logs de l'événement de déploiement de la version 2.4.7. Il trouve 81 secondes d'écart et présente ce lien comme une corrélation temporelle, pas comme une preuve.

En relisant le premier rapport, nous avons trouvé deux défauts, que nous avons corrigés et testés :

- Un décalage de deux heures sur les horodatages Docker : les chiffres du fuseau horaire étaient comptés dans les fractions de seconde. Le bug était invisible en CI, qui tourne en UTC.
- L'événement de déploiement sortait de la fenêtre de logs bornée. Le collecteur utilisait alors une heure statique, qui faussait la conclusion.

C'est pour cela que l'incident a été rejoué.

C2. Collecte minimale et masquage

Seules les lignes WARN et ERROR sont gardées. Les adresses e-mail, les jetons et les chaînes de connexion sont masqués, et le volume est limité (500 lignes lues, 40 exemples). Une ligne qui ressemble à une instruction (injection de prompt) est isolée et n'est jamais envoyée au modèle.

C3. Appel à Ollama

L'appel se fait en `POST /api/generate` en local, avec qwen3:14b. Le modèle a bien été utilisé, sans repli sur le mode simulé. Il a fallu deux réglages pour ce modèle, qui « réfléchit » avant de répondre :

- désactiver ce mode avec `think: false`, sans quoi il renvoyait un JSON vide ;
- porter le délai d'attente à 300 secondes, car il produit environ 9 tokens par seconde sur notre machine.

C4. Sortie structurée

Le prompt présente les preuves comme des données non fiables et impose un JSON avec : symptômes, hypothèses (confiance, preuves, preuves manquantes), contrôles non destructifs, limites et remédiation. Le code vérifie ensuite cette sortie. Une confiance trop élevée est plafonnée, et toute action sensible (rollback, redémarrage) passe obligatoirement par une approbation humaine.

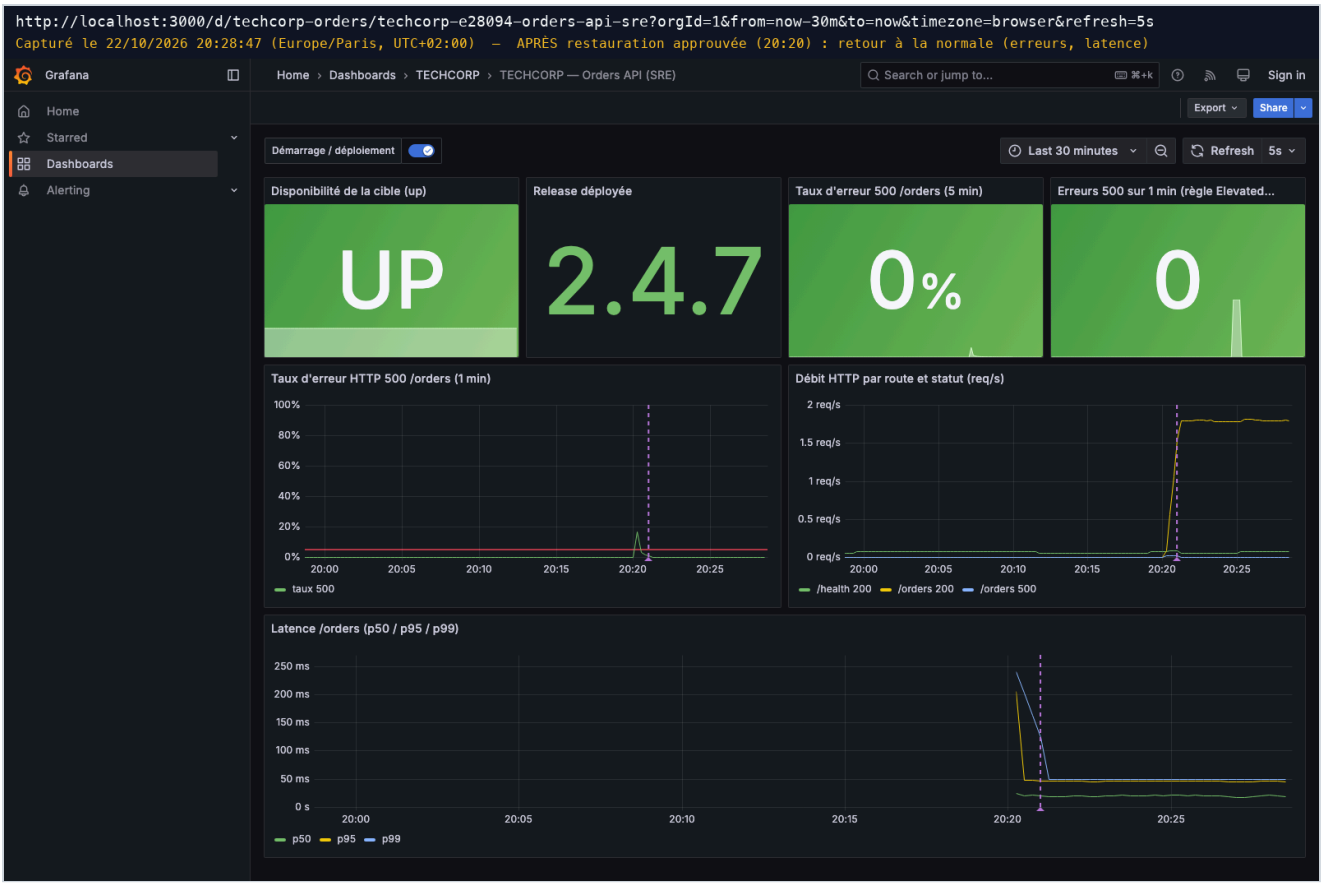
C5. Rapports et approbation

Les fichiers `reports/incident-report.md` et `reports/report.json` portent sur l'incident rejoué. L'action « Rollback vers 2.4.6 » a été approuvée par Paul Chavanon à 20:19:34, en tapant `APPROUVE 1`. Le script `restore.sh` refuse de s'exécuter sans cette trace, et l'outil n'exécute jamais d'action lui-même.

Mission D : revue SRE

D21 et D22. Pipeline et Grafana

Le pipeline rouge puis vert est présenté en A4, et Grafana avant et pendant l'incident en B2. Voici le retour à la normale, après la restauration approuvée :



20:28:47. Après restore.sh (20:20) : 0 % d'erreur sur 5 minutes, p95 revenu vers 47 ms. Le pic de 20:20 correspond aux dernières secondes de l'incident. Il n'y avait pas de trafic pendant l'attente de l'approbation.

Après la restauration, le détecteur répond OK, avec un code retour de 0.

D23. Calcul du détecteur

Les étapes du calcul sont les suivantes :

- récupérer les requêtes et les erreurs minute par minute sur une heure ;
- prendre une fenêtre courante de 5 minutes ;
- calculer z = (taux courant - moyenne) / max(écart-type ; 0,5 %) ;
- vérifier la double condition seuil et statistique.

Le plancher de 0,5 % évite de diviser par zéro quand l'historique est parfaitement plat. Les trois sorties réelles de B5 montrent la principale limite : une baseline polluée par un incident passé rend l'anormal « normal ». Pistes d'amélioration : médiane et MAD, baseline par tranche horaire, exclusion des périodes d'incident.

D24. Contester l'hypothèse de l'IA

Le rapport attribue l'incident à la version 2.4.7 avec 75 % de confiance. Cette confiance n'est pas justifiée :

- La seule preuve est une coïncidence de 81 secondes entre le déploiement et les erreurs.
- Aucun diff entre 2.4.6 et 2.4.7 n'a été examiné, et aucune métrique de la base n'est disponible.
- Un `db_connection_timeout` peut venir autant de la base ou du réseau que du code.
- Le modèle écrit qu'un z-score de 1,44 est « significatif », alors que notre seuil est de 3.
- Il propose un rollback conditionnel mal formulé.

Nous avons donc approuvé le rollback comme mesure réversible pour limiter l'impact, pas comme preuve de la cause. En laboratoire, la vraie cause est connue : `SIMULATE_FAILURE=true`.

D25. Plan de restauration

1. **Lever le doute** en lecture seule (diff, métriques de la base, autres routes).
2. **Rollback** avec un critère d'arrêt : si le taux d'erreur ne baisse pas après 5 minutes, on arrête et on escalade vers l'équipe base de données.
3. **Valider** : taux d'erreur sous 1 %, p95 normal, alertes résolues.
4. **Surveiller** pendant 30 minutes après le changement.
5. **Post-mortem** sans recherche de coupable, avec des actions concrètes : sonde readiness qui teste la base, baseline plus robuste.

Détail : `docs/plan-restauration.md`.

Livrables

Livrable demandé	Emplacement
Dépôt Git, commits lisibles, README	Dépôt GitHub ci-dessus, README avec les commandes de reproduction
Preuves pipeline en échec puis en succès	PR #2, `evidence/A4-*`
Captures Grafana et requêtes PromQL	`evidence/B2-*`, `evidence/D-grafana-apres.png`, `docs/observabilite.md`
Détecteur avec au moins 3 tests	`aiops/detector.py`, `tests/test_detector.py` (10 tests)
report.json, incident-report.md	`reports/`
Registre des risques	`docs/registre-risques.md`